1. CORE LANGUAGE SYNTAX, PRECEDENCE & CONTROL FLOW

Lectures 1–5 · Foundation Principles

DATA TYPES & LIMITS — MEMORY

Type	Bytes	Range & Behavior
`bool`	1	`true`(1), `false`(0); any non-zero=true
`char`	1	`'0'`=48, `'A'`=65, `'a'`=97; `'c'-'a'=2`
`int`	4	-2.14B to +2.14B. **Div truncates!**
`unsigned`	4	0 to 4.29B. Wrap: `0u-1u=4294967295`
`double`	8	~15 dig. `fabs(a-b)<1e-7`

#1 Trap — Truncating Division: `7/2=3`, `1/2=0`, `1/(i+1)=0` for $i \geq 1$. Cast before: `(double)a/b=3.5`. Note: `(double)(a/b)=3.0` (too late!).

Modulo % Rules: Integers only. Sign follows left: `-7 % 3 = -1`; `7 % -3 = 1`; `-7 % -3 = -1`.

PRECEDENCE & TRAPS — LOGIC

- **Precedence:** `() [] x++ x-- → ++x --x ! (type) sizeof` (R-to-L) → `* / %` → `+ -` → `<< >>` → `< <= > >=` → `== !=` → `&&` → `||` → `?:` → `= += -= *=`.
- **Prefix vs Postfix:** `y=++x` (increments *x* first, assigns new *x*); `y=x++` (assigns original *x*, then increments).
- **Chained Relational Trap:** `1 < x < 5` → `(1 < x) < 5` → `(0 or 1) < 5` is **ALWAYS TRUE**! Must write: `(1 < x && x < 5)`.
- **Assignment in if:** `if(x = 5)` assigns 5 and is always true! Must write `if(x == 5)`.
- **Short-Circuit:** In `A && B`, if *A* is false, *B* is skipped. Protects bounds: `if(i < n && A[i] > 0)`!

LOOPS & BRANCHING — FLOW

- **Dangling else:** Binds to closest unmatched `if`! Always use explicit braces `{ }`.
- **Switch:** Must be integral (`int`/`char`/`bool`). No `double`! Each case requires `break;` or falls through.
- **Loop Trips:** `for(i=0;i<n;i++)` (n trips, final i=n). `for(i=0;i<=n;i++)` (n+1 trips, final i=n+1).
- **Semicolon Trap:** `for(i=0;i<5;i++); { cout<<i; }` loops empty 5 times, then prints `5` once!
- **Jump:** `break` exits innermost loop/switch; `continue` jumps to update step. `do{...}while(c);` runs ≥ 1 time!

2. GORDON TRACING PROTOCOL (40%), ARRAY BOUNDARIES & FUNCTIONS

Question 2 Signature

TRACING PROTOCOL (40%) — TABLE

Rule: Column per variable + **Output**. Only write when value changes. Print when `cout` executes.

Iter	i	sum	a	B[i]	Output
init	0	0	1	11	–
iter 1	1	11	8	7	1 8
iter 2	2	18	14	6	2 14
iter 3	3	24	19	5	3 19
after	3	29	19	–	29 19 26
A[i]	4	–	–	G	Garbage (G)

Garbage (G) Rule: Array `A[4]` has indices `0..3`. When `for(i=0;i<4;i++)` ends, `i==4`. Printing `cout<<A[i];` is out-of-bounds → **Write "Garbage" or "G"**!

ARRAYS & C-STRINGS — MEMORY

- **Partial Init:** `int A[5]={10,20};` → A[0]=10, A[1]=20, A[2]=A[3]=A[4]=**0** (guaranteed zero, NOT garbage!).
- **Elements:** `int n = sizeof(A) / sizeof(A[0]);`
- **2D Matrix M[R][C]:** Rows 0..R-1, cols 0..C-1. Passing to function requires col size: `void f(double M[][4], int r)`.
- **C-String:** `char s[20];` **MUST** end with `'\0'`. `strlen(s)` excludes `'\0'`. Array size ≥ len+1.
- **Input:** `cin >> s;` stops at space. `cin.getline(s, 20);` reads line including spaces.

FUNCTIONS & FILE I/O — STREAMS

- **Pass-by-Value:** `void f(int x)` makes local copy; caller unchanged.
- **Pass-by-Reference:** `void f(int &x)` receives alias; **modifies caller**! Swap: `void swap(double &a, double &b){ double t=a; a=b; b=t; }`
- **Arrays:** Decay to pointer: `void f(double A[], int n)`. Modifies caller array directly! Always pass *n*.
- **File Streams (<fstream>):**
  - In: `ifstream fin("in.txt"); if(!fin) return 1;`
  - Loop: `while(fin >> val && n < MAX) A[n++] = val; fin.close();`
  - Out: `ofstream fout("out.txt"); fout << fixed << setprecision(3) << ans; fout.close();`

3. THE 60% WRITE-A-PROGRAM MASTER ALGORITHM BRIDGE

EXAM CODE TEMPLATES & EXPLANATIONS

1. Dynamic 1D & 2D Array Zero-Leak — new[]/delete[]

```
// 1D Array: alloc, check null, free with delete[]:
double *A = new double[n];
if (!A) { cout << "Error"; return 1; }
delete[] A; A = nullptr;
// 2D Array (R x C): alloc rows, then dealloc rows:
double **M = new double*[R];
for (int i = 0; i < R; i++) M[i] = new double[C];
for (int i = 0; i < R; i++) delete[] M[i];
delete[] M; M = nullptr;
```

2. Array Equality & Comparison — Tolerance & Break

```
// Compare element-by-element; break on mismatch:
bool match = true;
for (int i = 0; i < n; i++) {
  // Doubles tolerance: fabs(A[i]-B[i]) > 1e-7
  if (A[i] != B[i]) { match = false; break; }
}
if (match) cout << "Equal"; else cout << "Not equal";
```

3. In-Place Reversal & Merge — No Extra Mem

```
// In-place reverse: swap i with n-1-i up to n/2:
for (int i = 0; i < n / 2; i++) {
  double t = A[i];
  A[i] = A[n - 1 - i]; A[n - 1 - i] = t;
}
// Concatenate A and B (size n) into C (size 2n):
for (int i = 0; i < n; i++) { C[i] = A[i]; C[i+n] = B[i]; }
```

4. Min/Max & Safe Init — Never Init 0

```
// Init max with A[0] (fails if all negative!):
double maxV = A[0]; int maxIdx = 0;
for (int i = 1; i < n; i++) {
  if (A[i] > maxV) { maxV = A[i]; maxIdx = i; }
}
// Dot product & average: sum += A[i]*B[i]; avg=sum/n;
```

5. C-String Space Stripper — '\0' Re-seal

```
// In-place space removal (Gordon staple):
void remove_spaces(char s[]) {
  int j = 0; // write pointer
  for (int i = 0; s[i] != '\0'; i++) {
    if (s[i] != ' ') s[j++] = s[i];
  }
  s[j] = '\0'; // CRUCIAL: Must re-seal with null!
}
```

6. 2D Matrix Transpose & Sum — Nested Loops

```
// Transpose square matrix M[N][N]:
// j starts at i+1 to avoid re-swapping:
for (int i = 0; i < N; i++) {
  for (int j = i + 1; j < N; j++) {
    double t = M[i][j]; M[i][j] = M[j][i]; M[j][i] = t;
  }
}
```